

# CSA 0921 - PROGRAMMING IN JAVA

## CO1 - AT3 - PSEUDO CODE WRITING

NAME: VIDHUSHINI.T.S

REG. NO.: 192511061

**CO1-AT3-PSEUDOCODE WRITING** ← Back

**QUESTIONS**

**Q1**

Student Management System  
[BL3 – Apply]  
10 marks

**Q2**

2. Library Book Management  
[BL3 – Apply]  
10 marks

**Q3**

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

**Q4**

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

**Q5**

Vehicle Registration System  
[BL3 – Apply]  
10 marks

**Q6**

**Q1 Student Management System [BL3 – Apply]** 10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.  
OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

**Your Answer**

```
START
CLASS Student
    PRIVATE name
    PRIVATE rollNo
    PRIVATE marks
    PRIVATE average
    METHOD acceptDetails()
        INPUT name
        INPUT rollNo
        INPUT marks
    END METHOD
```

**CO1-AT3-PSEUDOCODE WRITING** ← Back

**QUESTIONS**

**Q1**

Student Management System  
[BL3 – Apply]  
10 marks

**Q2**

2. Library Book Management  
[BL3 – Apply]  
10 marks

**Q3**

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

**Q4**

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

**Q5**

Vehicle Registration System  
[BL3 – Apply]  
10 marks

**Q6**

**Q2 2. Library Book Management [BL3 – Apply]** 10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.  
OOP Concepts: Encapsulation | Getters & Setters | Objects

**Your Answer**

```
start
class book
    private title
    private author
    private ISBN
    private isavailable
    method setdetails(t,a,i)
        title<-t
        author<-a
        ISBN<-i
        isavailable<-true
```

## CO1-AT3-PSEUDOCODE WRITING

← Back

### QUESTIONS

Q1

Student Management System  
[BL3 – Apply]  
10 marks

Q2

2. Library Book Management  
[BL3 – Apply]  
10 marks

Q3

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

Q4

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

Q5

Vehicle Registration System  
[BL3 – Apply]  
10 marks

Q6

Shape Area Calculator using Polymorphism [BL3 – Apply]

### Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empld, name, basicPay, designation). Implement methods to:

- Calculate gross salary (basic + HRA + DA)
- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

### Your Answer

```
START
CLASS Employee
    PRIVATE empld, name, designation, basicPay
    PRIVATE HRA, DA, PF, tax, grossSalary, netSalary
    METHOD acceptDetails()
        INPUT empld, name, designation, basicPay
    END METHOD
    METHOD calculateGross()
        HRA <- basicPay*0.20
        DA <- grossSalary*0.10
        grossSalary <- basicPay+HRA+DA
```

## CO1-AT3-PSEUDOCODE WRITING

← Back

### QUESTIONS

Q1

Student Management System  
[BL3 – Apply]  
10 marks

Q2

2. Library Book Management  
[BL3 – Apply]  
10 marks

Q3

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

Q4

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

Q5

Vehicle Registration System  
[BL3 – Apply]  
10 marks

Q6

Shape Area Calculator using Polymorphism [BL3 – Apply]

### Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

### Your Answer

```
START
CLASS Shape
    METHOD calculateArea()
        PRINT "Area"
    END METHOD
END CLASS
CLASS Circle EXTENDS Shape
    radius
    OVERRIDE calculateArea()
        area <- 3.14*radius*radius
        PRINT area
```

## CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

## QUESTIONS

**Q1**  
Student Management System  
[BL3 – Apply]  
10 marks

**Q2**  
2. Library Book Management  
[BL3 – Apply]  
10 marks

**Q3**  
Employee Salary Calculation  
[BL3 – Apply]  
10 marks

**Q4**  
Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

**Q5**  
Vehicle Registration System  
[BL3 – Apply]  
10 marks

**Q6**  
Bank Account Hierarchy [BL4 – Analyze]  
10 marks

**Q5 Vehicle Registration System [BL3 – Apply]**

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

**Your Answer**

```
START
CLASS Vehicle
    regno, owner, fuelType
    METHOD displayDetails()
        PRINT regno, owner, fuelType
    END METHOD
END CLASS
CLASS TwoWheeler EXTENDS Vehicle
    engineCapacity
    OVERRIDE displayDetails()
        PRINT regno, owner, fuelType, engineCapacity
    END METHOD
END CLASS
```

## CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

## QUESTIONS

**Q1**  
Student Management System  
[BL3 – Apply]  
10 marks

**Q2**  
2. Library Book Management  
[BL3 – Apply]  
10 marks

**Q3**  
Employee Salary Calculation  
[BL3 – Apply]  
10 marks

**Q4**  
Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

**Q5**  
Vehicle Registration System  
[BL3 – Apply]  
10 marks

**Q6**  
Bank Account Hierarchy [BL4 – Analyze]  
10 marks

**Q6 Bank Account Hierarchy [BL4 – Analyze]**

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

**Your Answer**

```
START
CLASS BankAccount
    accno, holder, balance
    METHOD withdraw(amount)
        balance <- balance - amount
    END METHOD
END CLASS
CLASS SavingsAccount EXTENDS BankAccount
    OVERRIDE withdraw(amount)
        IF amount <= Balance THEN
```

## CO1-AT3-PSEUDOCODE WRITING

← Back

### QUESTIONS

**Q1**

Student Management System  
[BL3 – Apply]  
10 marks

**Q2**

2. Library Book Management  
[BL3 – Apply]  
10 marks

**Q3**

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

**Q4**

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

**Q5**

Vehicle Registration System  
[BL3 – Apply]  
10 marks

**Q6**

Bank Account Management [BL4 – Analyze]

### **Q7 Hospital Patient Record System [BL4 – Analyze]**

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

#### Your Answer

```
START
CLASS Patient
    PRIVATE patientId, name, diagnosis, prescription
    METHOD generateBill()
        PRINT "Patient Bill"
    END METHOD
END CLASS
CLASS InPatient EXTENDS Patient
    wardNo, admissionDate, dailyCharges
    OVERRIDE generateBill()
        bill <- dailyCharges*days
    PRINT bill
```

## CO1-AT3-PSEUDOCODE WRITING

← Back

### QUESTIONS

**Q1**

Student Management System  
[BL3 – Apply]  
10 marks

**Q2**

2. Library Book Management  
[BL3 – Apply]  
10 marks

**Q3**

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

**Q4**

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

**Q5**

Vehicle Registration System  
[BL3 – Apply]  
10 marks

### **Q8 E-Commerce Product Catalog [BL4 – Analyze]**

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

#### Your Answer

```
START
CLASS Product
    productId, name, price
    METHOD applyDiscount()
        PRINT price
    END METHOD
END CLASS
CLASS Electronics EXTENDS Product
    OVERRIDE applyDiscount()
        price <- price-(price*10/100)
    PRINT price
```

## CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

## QUESTIONS

Q1

Student Management System  
[BL3 – Apply]  
10 marks

Q2

2. Library Book Management  
[BL3 – Apply]  
10 marks

Q3

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

Q4

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

Q5

Vehicle Registration System  
[BL3 – Apply]  
10 marks

Q6

## Q9 University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure. Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

## Your Answer

```
START
CLASS Staff
    PRIVATE staffid, name
    PROTECTED salary
    METHOD calculateAllowance()
        PRINT "Allowance"
    END METHOD
END CLASS
CLASS TeachingStaff EXTENDS Staff
    OVERRIDE calculateAllowance()
```

## CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

## QUESTIONS

Q1

Student Management System  
[BL3 – Apply]  
10 marks

Q2

2. Library Book Management  
[BL3 – Apply]  
10 marks

Q3

Employee Salary Calculation  
[BL3 – Apply]  
10 marks

Q4

Shape Area Calculator using  
Polymorphism [BL3 – Apply]  
10 marks

Q5

Vehicle Registration System  
[BL3 – Apply]  
10 marks

## Q10 Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight,

SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

## Your Answer

```
START
CLASS SmartDevice
    METHOD executeCommand(cmd)
        PRINT cmd
    END METHOD
END CLASS
CLASS SmartLight EXTENDS SmartDevice
    OVERRIDE executeCommand(cmd)
        PRINT "Light"+cmd
    END METHOD
```